

Seeing Is Not Measuring: Tool-Augmented Metric Spatial Reasoning for Vision-Language Models

Clemens Grange Kai Glantz
Technical University of Munich

{clemens.grange, kai.glantz}@tum.de

Abstract

Vision-Language Models (VLMs) describe scenes well but reason poorly about metric 3D structure such as absolute distances, physical sizes, or egocentric directions. We present a modular, predictor-agnostic, tool-augmented framework that equips a small VLM (Qwen3.5-4B) with geometric tools: 3D object detection, metric depth estimation, and deterministic solvers for distance, size and bearing. Each object is detected in the camera frame of its own best view, and the tools use that frame’s pose to lift every detection into one shared world frame. Moving metric computation out of the model’s weights and into explicit solvers yields large gains on three of four ReVSI-Bench tasks: with a strong monocular detector (WildDet3D), absolute distance rises from 0.46 to 0.74 Mean Relative Accuracy (MRA), relative distance from 39.1% to 67.4%, and relative direction from a below-chance 25.9% to 73.4%. Because any detector can be swapped in behind the tool interface, comparing real detectors against ground-truth boxes separates perception error from reasoning error: orchestration costs only 0.03 MRA. Object size is bounded by the detector: the tools are near-exact on ground-truth boxes (0.97) yet the best real detector barely beats the no-tool baseline (0.61 vs. 0.58), because size reads straight off a box extent monocular detectors get wrong. Without a predefined recipe, the model already sequences the tools correctly on its own, matching a scripted pipeline on three of four tasks.

1. Introduction

Embodied and robotic agents operating in real environments need precise metric spatial understanding: how far apart two objects are, how large they are, and where they lie relative to the viewer. Vision-Language Models (VLMs) excel at qualitative scene description, yet remain unreliable at exactly this kind of metric 3D reasoning.

This bottleneck is documented by benchmarks such as VSI-Bench [23] and its corrected successor ReVSI-

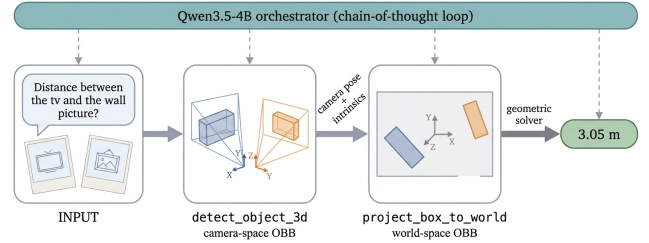


Figure 1. **Example of tool-augmented metric spatial reasoning.** The VLM orchestrates tools to answer a spatial distance question from video frames: `detect_object_3d` returns an oriented 3D box in the *camera* frame of each object’s own best view, `project_box_to_world` lifts those boxes into a single shared metric world frame using the frames’ camera poses, and a deterministic solver returns the surface-to-surface distance.

Bench [28]. On the ARKitScenes subset, Qwen3.5-4B [18] reaches only 0.46 Mean Relative Accuracy (MRA) on absolute distance, and smaller models fall further, to 0.21 (2B) and 0.00 (0.8B). Simply supplying the missing information does not help: appending the camera intrinsics to the prompt as text lowers MRA to 0.17. Metric geometry is better encapsulated behind something the model can call than verbalized into its context window.

We present a modular, predictor-agnostic framework (Fig. 1) that equips Qwen3.5-4B with geometric tools to detect camera-space 3D boxes, project them into a shared world coordinate system, and compute distances, sizes, and bearing angles, orchestrated through a chain-of-thought (CoT) tool-calling loop. Unlike prior tool-augmented work (Sec. 2), we ask whether a single, off-the-shelf small VLM can orchestrate such tools on its own, and when it fails, whether the fault lies in its tool-calling or in the underlying 3D perception. The perception backend swaps freely behind the same tool interface: a ground-truth (GT) oracle, real monocular 3D detectors (Cube R-CNN [3], OVMono3D [26], WildDet3D [11]), or a 2D grounding plus metric-depth pipeline (Grounding DINO [15] with Depth Pro [2]).

With a strong detector the tools improve the small model on all four tasks, though only marginally on object size (Sec. 4). The largest gain is on relative direction, where

the unaided model scores 25.9% against a chance rate of 29.3%, no better than a blind model that never sees the frames (28.3%); with tools it reaches 73.4%. Substituting the GT oracle behind the same tool chain then localizes the residual error: absolute distance climbs to 0.94 MRA, 0.03 below the *geometric ceiling*, the score the same tools reach when a script rather than the VLM calls them. This leaves the monocular detector as the binding constraint, and it binds hard. On object size the tools are near-exact on ground-truth boxes (0.97), yet the best real detector only just clears the 0.58 no-tool baseline (0.61); on absolute distance every backend but one falls *below* the 0.46 baseline. Relative direction is the one task limited by the model rather than perception: it caps at 80% against a 100% ceiling, skipping the tools on 23% of questions.

Contributions.

- A **predictor-agnostic tool framework** that lifts camera-space detections into a shared, oriented world frame using the camera poses, letting a small VLM reason metrically across views. The same tool chain runs on monocular 3D detectors or on 2D grounding with metric depth.
- An **orchestration-perception error decomposition** over four ReVSI-Bench tasks. Comparing each detector backend, behind a fixed tool chain, against a ground-truth oracle and a VLM-free geometric ceiling separates the two error sources and attributes the residual error on three tasks to the detector.
- An **analysis of tool-use behavior**. The off-the-shelf VLM already discovers correct tool sequencing without an explicit recipe; what it lacks is the discipline to invoke the tools on questions that appear answerable by inspection, which we identify as the target for fine-tuning.

2. Related Work

Benchmarks for spatial reasoning. Early spatial-intelligence evaluations for VLMs focused on 2D relationships (e.g., “left of”, “inside”) on datasets like GQA [12], missing the metric 3D understanding embodied interaction needs. VSI-Bench [23] and its corrected successor ReVSI-Bench [28] instead probe metric spatial reasoning over multi-view video of indoor scans, and MMSI-Video-Bench [14] broadens the task suite. MV-RoboBench [9] makes the multi-view setting explicit for robotic manipulation and finds that single-view spatial competence does not reliably transfer to multi-camera scenes. We evaluate on ReVSI-Bench because it provides, per question, the camera poses and intrinsics our tools need plus the ground-truth 3D boxes for an oracle detector; benchmarks without camera calibration are out of reach for this framework.

Baking spatial ability into the model. A broad wave of work trains 3D spatial competence into the VLM itself: supervised fine-tuning (MM-Spatial [7], Visual Spatial Tuning [24], SpatialLadder [13], Spatial-MLLM [22]), spatial-reward RL (SpatialThinker [1]), 3D-reconstruction architectures (VLM-3R [8]), and scaling studies (Cambrian-S [25], Scaling Spatial Intelligence [4]). All change the model’s weights; we instead leave it untouched and ask how far a small, in-context VLM gets through tool orchestration alone.

Tool-augmented and agentic VLMs. Several works equip VLMs with external tools to bypass unreliable direct reasoning, interleaving reasoning with tool calls as in ReAct [27] rather than baking the calls into the weights as in Toolformer [19]: SpaceTools [5] uses two-phase interactive RL to teach a VLM to compose pointing and depth tools on a single image. Agentic frameworks (Visual Programming [10], ViperGPT [20]) compile queries into executable programs. GCA [6] and SpatialPIN [16] combine prompting with geometric constraints and 3D priors, and ViSRA [17] pairs a VLM with camera-lifted point clouds to build an explicit 3D scene. We instead target multi-view, video-based geometry with a training-free orchestration loop across frames.

3. Method

3.1. Dataset

We evaluate on the ARKitScenes subset of ReVSI-Bench [28] across four question categories: absolute distance (401 questions), object size (423), relative distance (215), and relative direction (290), each question coming with 16 candidate frames sampled from the scene’s video. Unlike VSI-Bench [23], ReVSI-Bench verifies the queried objects are actually visible across these frames, guaranteeing the question is answerable in the first place. We merge this with the original ARKitScenes camera intrinsics and extrinsics for all 16 frames and the ground-truth 3D boxes of the objects relevant to each question, provided by ReVSI-Bench. Numeric tasks (distance, size) use the benchmark’s Mean Relative Accuracy, $MRA = \frac{1}{|\mathcal{C}|} \sum_{\theta \in \mathcal{C}} \mathbf{1}[|\hat{x} - x|/x < 1 - \theta]$ over the confidence thresholds $\mathcal{C} = \{0.5, 0.55, \dots, 0.95\}$; classification tasks (relative distance, direction) use exact-letter accuracy.

3.2. Frame Selection

Rather than handing all 16 frames to the VLM, we select only the frames a question needs: two for a distance comparison, one for object size. For each relevant object we project its ground-truth 3D box corners into every candidate frame using that frame’s camera pose and intrinsics, and keep the frame(s) where the projected corners stay furthest inside the image bounds, i.e. where the object is most visible. Passing all 16 frames per question is computationally

impractical, and the 3D detector fails outright on an object a frame does not clearly show. Letting the VLM pick its own frames proved unreliable, so selection happens up front; it is therefore load-bearing rather than preprocessing, and Sec. 5 returns to its reliance on ground-truth boxes.

3.3. Tool-Use Loop

At each turn the VLM (Qwen3.5-4B) receives the selected frame(s), the question, and a system prompt listing the available tools, their JSON signatures, a pipeline-specific hint on the order to chain them, and the answer format (Sec. B). Rather than reading metric quantities off the pixels, it acts as an orchestrator (Fig. 1): it reasons in a chain-of-thought (CoT) loop and calls tools one at a time via JSON payloads wrapped in `<tool_call>` tags, passing only object identifiers (e.g., "sofa"). Coordinates and matrices stay cached in the host environment, so the context never fills with numbers and the model never does the arithmetic itself.

3.4. Toolsets and Pipelines

A per-task toolset (Tab. A4, Sec. C) exposes the tools the question needs and hints at the order to chain them.

Absolute Distance. The baseline `gdino_depthpro_abs_dist` pipeline detects a 2D box (Grounding DINO [15]), estimates depth (Depth Pro [2]), and computes center-to-center distance. Since ReVSI-Bench ground truth measures surface-to-surface distance, `bbox_3d_abs_dist` instead detects 3D camera-space OBBs (`detect_object_3d`), lifts them to world space (`project_box_to_world`), and computes the minimum surface distance (`calculate_object_distance`). The choice of geometry accounts for most of the achievable accuracy: on GT boxes, center-to-center distance has an MAE of 0.644 m against the benchmark answer, versus 0.077 m for surface-to-surface.

Object Size. Object size is the longest dimension of the oriented 3D extent. The `bbox_3d_size` pipeline detects the camera-space box and extracts its longest side (`calculate_object_size`); no world-frame projection is needed since extent is rotation-invariant. The `gdino_depthpro_size` baseline instead back-projects the 2D box, which measures the object’s apparent projection and ignores its extent along the viewing direction.

Relative Distance and Direction. Relative distance ranking (`bbox_3d_reldist`) generalizes the absolute distance chain across candidates to find the closest/farthest option. Relative direction tracking (`bbox_3d_reldir`) projects viewer, facing, and target into a unified world space and classifies the egocentric bearing angle θ into 3-way or 4-way sectors.

3.5. Predictor-Agnostic Detector Backends

Swapping the backend behind `detect_object_3d` changes only where the camera-space boxes come from, never the tool-calling structure. A **GT oracle** isolates orchestration; **monocular 3D detectors** (Cube R-CNN [3], OVMono3D [26], WildDet3D [11]) substitute real predictions, cached for reproducibility.

4. Experiments

We run on the full ReVSI-Bench ARKitScenes splits of Sec. 3, using cached backend predictions for determinism; the orchestrator is Qwen3.5-4B, decoded greedily (Sec. A).

4.1. Baselines and Model Scale

Native (no-tool) VLMs degrade sharply below 4B (Tab. 1), leaving Qwen3.5-4B as the only usable baseline; even there `+intrinsic`s costs it 0.29 MRA.

4.2. Main Results

Under the GT oracle, our framework reaches 0.94/0.97 VLM MRA against a tool ceiling of 0.97/1.00: orchestration costs 0.03 MRA, and the rest is perception.

Real detectors bear this out: every backend but one falls below the 0.46 no-tool baseline. OVMono3D reaches 0.26, Cube R-CNN 0.32, and the 2D+depth pipeline 0.32, the last undone by depth outliers of up to 10 187 m. A bad box is worse than no box at all (Sec. E); where a backend misses objects, the model’s fallback guess can even push its MRA above its own ceiling (Sec. A). Only WildDet3D pays off, lifting distance from 0.46 to 0.74 at full coverage.

Object size is bounded by the detector alone. The tools are near-exact given good boxes (0.97 against a 1.00 ceiling), yet the best real detector barely converts that into a gain: WildDet3D reaches 0.61 against the 0.58 baseline, because size reads straight off the box extent and WildDet3D’s longest side is a median 14.3% off. Distance tolerates a noisy box, since surface distance is dominated by the separation of the centers; size does not.

4.3. Multiple-Choice Tasks

Relative Distance. Tool calling lifts accuracy from 39.1% (visual) to 67.4% (WildDet3D) and 92.1% (GT), 2.8 points below the GT ceiling (94.9%), which itself falls short of 100% because the ground truth scores against the closest instance of a name (Sec. 5).

Relative Direction. Chance is 29.3% (149 three-option and 141 four-option questions). Both no-tool baselines sit at or below it, and the visual baseline (25.9%) does not beat the blind one (28.3%), so the frames carry no orientation signal the VLM can use. The bearing tool raises accuracy to 80.0% (GT) and 73.4% (WildDet3D). The two ceilings

No-tool baselines, absolute distance (MRA $\uparrow$)			
Model	visual	blind	+intrinsic
Qwen3.5-0.8B	0.00	0.00	0.02
Qwen3.5-2B	0.21	0.00	0.03
Qwen3.5-4B	0.46	0.29	0.17

Table 1. Native (no-tool) VLMs on absolute distance. Only the 4B model is a usable baseline; passing camera intrinsics as text does not help.

Method	MRA $\uparrow$	Ceiling $\uparrow$
Absolute distance (401 Q)		
No tools (visual)	0.46	–
Tools, 2D + depth	0.32	0.29
Tools, 3D boxes (OVMono3D)	0.26	0.26
Tools, 3D boxes (Cube R-CNN)	0.32	0.28
Tools, 3D boxes (WildDet3D)	0.74	0.74
Tools, 3D boxes (GT oracle)	0.94	0.97
Object size (423 Q)		
No tools (visual)	0.58	–
Tools, 2D + depth	0.34	0.34
Tools, 3D boxes (Cube R-CNN)	0.34	0.34
Tools, 3D boxes (WildDet3D)	0.61	0.62
Tools, 3D boxes (GT oracle)	0.97	1.00

Table 2. Absolute distance and object size results (ReVSI-Bench ARKitScenes); the toolset behind each row is given in Tab. A4. “Ceiling” runs the geometric tools directly on the boxes, without a VLM, so it measures what the perception backend allows before any orchestration.

Method	Acc. $\uparrow$	Parse	Ceiling
Relative distance (215 Q, chance 25.0%)			
No tools (blind)	30.2%	100.0%	–
No tools (visual)	39.1%	100.0%	–
Tools, 3D boxes (WildDet3D)	67.4%	100.0%	69.3%
Tools, 3D boxes (GT oracle)	92.1%	100.0%	94.9%
Relative direction (290 Q, chance 29.3%)			
No tools (blind)	28.3%	100.0%	–
No tools (visual)	25.9%	100.0%	–
Tools, 3D boxes (WildDet3D)	73.4%	98.3%	89.0%
Tools, 3D boxes (GT oracle)	80.0%	96.6%	100.0%

Table 3. Multiple-choice results on ReVSI-Bench. “Parse” is the share of runs emitting a usable option letter (Sec. A); unparseable answers score wrong. Both no-tool baselines parse at 100%, so their at-or-below-chance relative-direction accuracy is a real reasoning failure and not a formatting artifact.

separate the error sources: perception costs 11 points here (100.0 $\rightarrow$ 89.0%) against 25.6 on relative distance, since a bearing reads object centers and never their extents (Sec. G). Orchestration costs more, 15.6 points below the WildDet3D ceiling, because on 23% of questions the model calls no tool and answers directly (Sec. D).

4.4. No-Recipe Autonomous Probe

We expose all tools at once behind a generic prompt that names them but prescribes no order, over 40 questions from each of the four tasks, with the most generous of the scripted budgets (20 steps, 2048 tokens; Sec. A).

Reaching a working autonomous loop meant fixing the tools, not the prompt. The original error strings were terse (“No 3D box found for [tv, sink]”), so a mis-ordered call told the model nothing about what to do next and it simply re-

Method	MRA $\uparrow$		Acc. $\uparrow$	
	Dist.	Size	R. dist	R. dir
No recipe (generic prompt)	0.94	0.97	92.5	70.0
+ few-shot order examples	0.89	1.00	80.0	75.0
Scripted recipe (Tabs. 2 and 3)	0.94	0.97	92.1	80.0

Table 4. Autonomous probe: every tool exposed at once, no per-task recipe (160 questions, 40 per task, GT backend). Distance and size are scored by MRA, the two relative tasks by accuracy. The scripted-recipe row repeats the GT-oracle results of Tabs. 2 and 3, measured on the full splits rather than on this subsample.

tried. Rewriting each error to name the missing step removes most of that looping: in an 80-question ablation it cuts tool errors from 138 to 36 and raises distance from 0.58 to 0.88 MRA (Sec. H); all runs here use the rewritten errors. With that feedback the recipe becomes unnecessary (Tab. 4): the generic prompt reaches 0.94 and 0.97 MRA on distance and size and 92.5% on relative distance, matching the scripted pipeline on all three. Few-shot ordering examples do not improve on it: they gain on size and relative direction but cost 0.05 MRA on distance and 12.5 points of relative-distance accuracy. Relative direction again lags at 70.0%, where the model skips the tools. Routing capability is already present; the discipline to invoke the tools is not.

5. Conclusion and Next Steps

Conclusion. Encapsulating 3D geometry behind modular tools lets a small VLM perform multi-view metric spatial reasoning it cannot do from pixels alone. Comparing each detector against a ground-truth oracle attributes the residual error to monocular 3D perception: orchestration costs only 0.03 MRA, while a weak detector is worse than no tool at all.

Limitations. Our framework selects one “best frame” per object *using the ground-truth boxes*, and never triangulates across views. The no-tool baselines also see all 16 frames while the tool runs see only the selected one or two, so part of the gain may come from framing rather than from the tools; the GT-oracle numbers are invariant to the frame chosen. For relative distance we map each name to a single predicted box (5.1% ceiling penalty). All three detectors are trained on Omni3D [3], which contains the ARKitScenes training split; our 161 scenes come from the validation fold, so none is seen in training, but perception stays in-domain.

Next steps. Frame selection is the first oracle to remove: the visibility score should become a tool the model calls, scored by the open-vocabulary 2D detector we already expose. A feed-forward pose predictor [21] would remove the second oracle, letting the chain run on uncalibrated video. Beyond that: better box extents, multi-frame triangulation, instance-aware routing, and fine-tuning.

Acknowledgments

We thank our supervisor, Bartłomiej Baranowski, for his guidance and feedback throughout this project.

References

- [1] Hunar Batra, Haoqin Tu, Hardy Chen, Yuanze Lin, Cihang Xie, and Ronald Clark. Spatialthinker: Reinforcing scene graph-grounded spatial reasoning via dense rewards. *arXiv preprint arXiv:2511.07403*, 2025. 2
- [2] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R. Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. *arXiv preprint arXiv:2410.02073*, 2024. Apple Depth Pro. 1, 3
- [3] Garrick Brazil, Abhinav Kumar, Julian Straub, Nikhila Ravi, Justin Johnson, and Georgia Gkioxari. Omni3d: A large benchmark and model for 3d object detection in the wild. In *CVPR*, 2023. Cube R-CNN. 1, 3, 4
- [4] Zhongang Cai, Ruisi Wang, Chenyang Gu, Fanyi Pu, Junxiang Xu, Yubo Wang, Wanqi Yin, Zhitao Yang, Chen Wei, Qingping Sun, Tongxi Zhou, Jiaqi Li, Hui En Pang, Oscar Qian, Yukun Wei, Zhiqian Lin, Xuanke Shi, Kewang Deng, Xiaoyang Han, Zukai Chen, Xiangyu Fan, Hanming Deng, Lewei Lu, Liang Pan, Bo Li, Ziwei Liu, Quan Wang, Dahua Lin, and Lei Yang. Scaling spatial intelligence with multi-modal foundation models. In *CVPR*, 2026. 2
- [5] Siyi Chen, Mikaela Angelina Uy, Chan Hee Song, Faisal Ladhak, Adithyavairavan Murali, Qing Qu, Stan Birchfield, Valts Blukis, and Jonathan Tremblay. Spacetools: Tool-augmented spatial reasoning via double interactive rl. In *CVPR*, 2026. arXiv:2512.04069. 2
- [6] Zeren Chen, Xiaoya Lu, Zhijie Zheng, Pengrui Li, Lehan He, Yijin Zhou, Jing Shao, Bohan Zhuang, and Lu Sheng. Geometrically-constrained agent for spatial reasoning. *arXiv preprint arXiv:2511.22659*, 2025. 2
- [7] Erik Daxberger, Nina Wenzel, David Griffiths, Haiming Gang, Justin Lazarow, Gefen Kohavi, Kai Kang, Marcin Eichner, Yinfei Yang, Afshin Dehghan, and Peter Grasch. Mm-spatial: Exploring 3d spatial understanding in multimodal llms. In *ICCV*, 2025. 2
- [8] Zhiwen Fan, Jian Zhang, Renjie Li, Junge Zhang, Runjin Chen, Hezhen Hu, Kevin Wang, Huaizhi Qu, Shijie Zhou, Dilin Wang, Zhicheng Yan, Hongyu Xu, Justin Theiss, Tianlong Chen, Jiachen Li, Zhengzhong Tu, Zhangyang Wang, and Rakesh Ranjan. Vlm-3r: Vision-language models augmented with instruction-aligned 3d reconstruction. *arXiv preprint arXiv:2505.20279*, 2025. 2
- [9] Zhiyuan Feng, Zhaolu Kang, Qijie Wang, Zhiying Du, Jiongrui Yan, Shubin Shi, Chengbo Yuan, Huizhi Liang, Yu Deng, Qixiu Li, Rushuai Yang, Arctanx An, Leqi Zheng, Weijie Wang, Shawn Chen, Sicheng Xu, Yaobo Liang, Jialong Yang, and Baining Guo. Seeing across views: Benchmarking spatial reasoning of vision-language models in robotic scenes. *arXiv preprint arXiv:2510.19400*, 2025. 2
- [10] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14953–14962, 2023. 2
- [11] Weikai Huang, Jieyu Zhang, Sijun Li, Taoyang Jia, Jiafei Duan, Yunqian Cheng, Jaemin Cho, Matthew Wallingford, Rustin Soraki, Chris Dongjoo Kim, Shuo Liu, Donovan Clay, Taira Anderson, Winson Han, Ali Farhadi, Bharath Hariharan, Zhongzheng Ren, and Ranjay Krishna. Wilddet3d: Scaling promptable 3d detection in the wild. *arXiv preprint arXiv:2604.08626*, 2026. 1, 3
- [12] Drew A. Hudson and Christopher D. Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 2
- [13] Hongxing Li, Dingming Li, Zixuan Wang, Yuchen Yan, Hang Wu, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. Spatialladder: Progressive training for spatial reasoning in vision-language models. *arXiv preprint arXiv:2510.08531*, 2025. 2
- [14] Jingli Lin, Runsen Xu, Shaohao Zhu, Sihan Yang, Peizhou Cao, Yunlong Ran, Miao Hu, Chenming Zhu, Yiman Xie, Yilin Long, Wenbo Hu, Dahua Lin, Tai Wang, and Jiangmiao Pang. Mmsi-video-bench: A holistic benchmark for video-based spatial intelligence. *arXiv preprint arXiv:2512.10863*, 2025. 2
- [15] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *ECCV*, 2024. 1, 3
- [16] Chenyang Ma, Kai Lu, Ta-Ying Cheng, Niki Trigoni, and Andrew Markham. Spatialpin: Enhancing spatial reasoning capabilities of vision-language models through prompting and interacting 3d priors. In *NeurIPS*, 2024. arXiv:2403.13438. 2
- [17] Tingshu Mou, Jiabo He, Renying Wang, Ce Liu, Hao Yang, Tiegua Zhang, Jingjing Chen, and Xingjun Ma. Visra: A video-based spatial reasoning agent for multi-modal large language models. *arXiv preprint arXiv:2605.10106*, 2026. 2
- [18] Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, 2026. 1
- [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023. arXiv:2302.04761. 2
- [20] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11888–11898, 2023. 2
- [21] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *CVPR*, 2025. arXiv:2503.11651. 4
- [22] Diankun Wu, Fangfu Liu, Yi-Hsin Hung, and Yueqi Duan. Spatial-mlm: Boosting mllm capabilities in visual-based spatial intelligence. *arXiv preprint arXiv:2505.23747*, 2025. 2

- [23] Jihan Yang, Shusheng Yang, Anjali Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in space: How multimodal large language models see, remember, and recall spaces. *arXiv preprint arXiv:2412.14171*, 2024. VSI-Bench. [1](#), [2](#)
- [24] Rui Yang, Ziyu Zhu, Yanwei Li, Jingjia Huang, Shen Yan, Siyuan Zhou, Zhe Liu, Xiangtai Li, Shuangye Li, Wenqian Wang, Yi Lin, and Hengshuang Zhao. Visual spatial tuning. *arXiv preprint arXiv:2511.05491*, 2025. [2](#)
- [25] Shusheng Yang, Jihan Yang, Pinzhi Huang, Ellis Brown, Zihao Yang, Yue Yu, Shengbang Tong, Zihan Zheng, Yifan Xu, Muhan Wang, Daohan Lu, Rob Fergus, Yann LeCun, Li Fei-Fei, and Saining Xie. Cambrian-s: Towards spatial supersensing in video. *arXiv preprint arXiv:2511.04670*, 2025. [2](#)
- [26] Jin Yao, Hao Gu, Xuweiyi Chen, Jiayun Wang, and Zezhou Cheng. Open vocabulary monocular 3d object detection. *arXiv preprint arXiv:2411.16833*, 2024. OVMono3D. [1](#), [3](#)
- [27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023. [arXiv:2210.03629](#). [2](#)
- [28] Yiming Zhang, Jiacheng Chen, Jiaqi Tan, Yongsen Mao, Wenhui Chen, and Angel X. Chang. Revsi: Rebuilding visual spatial intelligence evaluation for accurate assessment of vlm 3d reasoning. In *International Conference on Machine Learning (ICML)*, 2026. [arXiv:2604.24300](#). [1](#), [2](#)

Seeing Is Not Measuring: Tool-Augmented Metric Spatial Reasoning for Vision-Language Models

Supplementary Material

A. Implementation Details

Orchestrator and decoding. Every tool-use run drives the same orchestrator, Qwen3.5-4B in `bfloat16`, with greedy decoding (`do_sample=False`); no sampling temperature is involved, so a run reproduces its own trace. Each selected frame enters the VLM at a budget of 16–144 visual tokens ($\approx 112^2\text{--}336^2$ px). The generation budget is 1024 new tokens per turn on the numeric tasks and 2048 on the multiple-choice and autonomous runs, so that a long chain-of-thought cannot truncate mid-tool call.

Step caps. The tool-use loop is capped per task, always with headroom over the minimal chain (Sec. F): 5 steps for object size (minimal 2), 10 for absolute distance (minimal 5), 14 for relative direction (minimal 7), and 20 for relative distance (minimal 14) and the autonomous probe. A run that hits the cap is scored on whatever answer it has produced, i.e. as a miss unless it already emitted one.

Answer parsing. Numeric answers are the last number in the final message; an unparseable answer scores $\text{MRA} = 0$ (a miss) but contributes no metre value, so it is excluded from the error statistics of Fig. A4. Multiple-choice answers are the option letter, or the uniquely matching option name if no bare letter is emitted; the resulting parse rates are the *Parse* column of Tab. 3.

Cached backends and coverage. `detect_object_3d` reads pre-computed camera-space boxes, so every backend is deterministic and the tool-calling structure is identical across them (Sec. 3). A backend that has no box for a queried name returns an error string instead, and the model then answers from the frames alone. Table A1 gives the resulting coverage on the absolute-distance split. This is the mechanism behind the two inversions in the main results. Cube R-CNN misses 8.2% of objects, and on the 15% of questions where a box is missing the VLM falls back to guessing, which beats the zero MRA the tool ceiling records for a missing box, so its VLM MRA (0.32) *exceeds* its own ceiling (0.28); the 2D grounding front-end inverts for the same reason (0.32 against a 0.29 ceiling).

Backend	Objects found	Questions fully covered
GT Oracle	100.0% (802/802)	100.0% (401/401)
OVMono3D	100.0% (802/802)	100.0% (401/401)
Cube R-CNN	91.8% (736/802)	85.0% (341/401)
WildDet3D	100.0% (802/802)	100.0% (401/401)
Grounding DINO (2D)	93.6% (751/802)	87.8% (352/401)

Table A1. Detector coverage on the absolute-distance split (401 questions, 802 queried object instances). *Objects found*: the backend returns a box for the queried name. *Questions fully covered*: boxes exist for *both* objects, i.e. the tool chain can run end-to-end. Cube R-CNN and the 2D grounding front-end miss objects, which is why their VLM MRA can exceed their own ceiling; OVMono3D is fully covered yet still the weakest backend (Tab. 2), so coverage and accuracy are independent failure modes.

B. System Prompts and Toolset Hints

Every run assembles its system prompt from a single template (Listing A1): a fixed preamble, the JSON schemas of the tools exposed for that pipeline, the pipeline-specific *hint* (the recipe), a one-tool-at-a-time calling convention, and the task-specific answer format. Only the tool subset, the task/answer-format strings, and the hint change between pipelines; the scaffold is identical. Table A2 lists the per-pipeline task and answer-format strings, and Listings A2 and A3 give two hints verbatim. The other four (2D+depth distance, object size, relative distance, relative direction) follow the same detect $\rightarrow$ project $\rightarrow$ measure shape and are omitted for space; Tab. A4 gives each pipeline’s tool chain.

Pipeline	task / answer_format
bbox_3d_abs_dist	distances between objects
gdino_depthpro_abs_dist	<i>a single number in metres, e.g.: 1.5</i>
bbox_3d_size	the size (longest dimension) of objects
gdino_depthpro_size	<i>a single number in centimetres, e.g.: 120</i>
bbox_3d_reldist	which of several candidate objects is closest or farthest to a reference object <i>the letter (A, B, C, or D)</i>
bbox_3d_reldir	the egocentric direction (left-/right/back, or a front/back-left/right quadrant) of a target from a viewer <i>the letter (A, B, C, or D)</i>
autonomous_3d	spatial questions about objects (all four tasks mixed) <i>the exact format the question asks for</i>

Table A2. Per-pipeline task and answer_format strings substituted into the scaffold of Listing A1. These reuse the same tool interface across numeric (MRA) and multiple-choice tasks.

Listing A1. Prompt scaffold (build_system_prompt). {tools} is the pretty-printed JSON of the exposed tool schemas; {hint}, {task} and {answer_format} are filled per pipeline.

```
You are a precise spatial measurement assistant.
You answer questions about {task} by calling tools.

Available tools:
{tools}

{hint}

Call ONE tool at a time and wait for its result before
calling the next tool.
To call a tool, use this exact format:
<tool_call>{"name": "tool_name",
  "arguments": {"arg": "value"}}</tool_call>

When you have computed the answer, output it as
{answer_format}
```

Listing A2. Hint for bbox_3d_abs_dist (absolute distance). The recipe enforces the camera→world lift before any metric comparison.

```
detect_object_3d returns a box in CAMERA space,
measured from that object's own camera, so two
such boxes cannot be compared until each is
lifted to world space.
Every tool takes ONLY object names -- never copy
coordinates between calls.
Follow this exact sequence; do not repeat a step:
1. detect_object_3d for the first object
   -> camera-space box
2. project_box_to_world for the first object
   -> world box
3. detect_object_3d for the second object
   -> camera-space box
4. project_box_to_world for the second object
5. calculate_object_distance (two names)
   -> distance in metres
Call detect_object_3d and project_box_to_world
```

exactly once per object, then output the number.

Listing A3. Hint for autonomous_3d (no-recipe probe, Sec. 4). The full 5-tool union with no step sequence: the model must route and order the tools itself.

You have tools to detect an object's 3D box, lift a box into world coordinates, measure an object's size, measure the distance between two objects, and compute the egocentric direction of one object from a viewer facing another. Decide which tools to call, and in what order, to answer THIS question -- not every tool is needed for every question. First read the question and identify what it asks: one object's size, the distance between two objects, which listed object is closest or farthest to a reference, or the direction of an object relative to a viewer. Use the tools to compute the answer rather than guessing from the images. Answer in the exact format the question asks for.

C. Tool Library and Geometric Logic

All metric computation lives inside the tools; the orchestrator only passes object *names*, and each tool reads the cached geometry for that name from the execution context. This section gives the exact math behind the four geometric solvers, plus how detect_object_3d turns pixels into a metric box in the first place. Let a detected oriented box be (c, R, e) with center $c \in \mathbb{R}^3$, rotation $R \in \text{SO}(3)$ (columns are the box axes), and full extent $e \in \mathbb{R}_{>0}^3$.

Camera-space detection (detect_object_3d). Unlike the solvers below, this tool is a predictor-agnostic wrapper around a monocular 3D detector (Cube R-CNN, OV-Mono3D, WildDet3D) or the GT oracle. The real detectors need the frame's camera *intrinsics*: we rescale and re-orient the native sensor matrix to the upright display frame the model actually sees, $K_{\text{eff}} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ (compute_keff), and pass K_{eff} into the model's forward pass so it can convert its 2D detection and relative depth into a metric camera-space box. This is the only place intrinsics enter the pipeline; downstream, project_box_to_world uses only extrinsics.

World projection (project_box_to_world). detect_object_3d returns a box in the *camera* frame of that object's best frame. Lifting it to world space needs only that frame's *extrinsics*: inverting the stored world-to-camera pose $(R_{w \rightarrow c}, t_{w \rightarrow c})$ gives the camera-to-world rigid transform $(R_{c \rightarrow w}, t_{c \rightarrow w}) \in \text{SE}(3)$, and

$$c_w = R_{c \rightarrow w} c_c + t_{c \rightarrow w}, \quad R_w = R_{c \rightarrow w} R_c, \quad e_w = e_c, \quad (1)$$

i.e. the extent is invariant (the lift is rigid). This is what makes two boxes from *different* cameras comparable; without it a distance between them is meaningless. No pixel is back-projected here, so the camera *intrinsics* play no role in this step.

Surface-to-surface distance (calculate_object_distance). ReVSI ground truth measures the distance between the nearest *surfaces*, not centers. We approximate it by subtracting each oriented box’s half-extent, projected onto the center-separation direction, from the center distance. With $\hat{\mathbf{d}} = (\mathbf{c}_b - \mathbf{c}_a) / \|\mathbf{c}_b - \mathbf{c}_a\|$,

$$\text{proj}(\text{box}) = |R^\top \hat{\mathbf{d}}| \cdot \frac{1}{2} \mathbf{e}, \quad (2)$$

$$d_{\text{surf}} = \max(0, \|\mathbf{c}_b - \mathbf{c}_a\| - \text{proj}(\text{box}_a) - \text{proj}(\text{box}_b)). \quad (3)$$

On GT boxes this reduces MAE against the benchmark from 0.644 m (center-to-center) to 0.077 m (71% of samples within 0.1 m).

Object size (calculate_object_size). Size is the longest side of the oriented box, in centimeters: $\text{size}_{\text{cm}} = 100 \cdot \max_i e_i$. Because the extent is rigid-invariant (above), no world lift is needed and the size pipeline is the minimal two-step chain `detect_object_3d`→`calculate_object_size`.

Egocentric bearing (relative_direction). Given three world centers projected to the ground plane (heights dropped), namely viewer *a*, facing *b* and target *c*, the facing and target vectors are

$$\mathbf{f} = \begin{cases} \mathbf{b} - \mathbf{a} & \text{mode} = \text{toward} \\ \mathbf{a} - \mathbf{b} & \text{mode} = \text{away} \end{cases}, \quad \mathbf{t} = \mathbf{c} - \mathbf{a}, \quad (4)$$

and the signed bearing (positive $\Rightarrow$ target on the *left*) is

$$\theta = \text{atan2}(f_x t_y - f_y t_x, \mathbf{f} \cdot \mathbf{t}). \quad (5)$$

It is classified into the answer’s direction words. 3-way: BACK if $|\theta| \geq 135^\circ$, else LEFT/RIGHT by sign. 4-way: FRONT if $|\theta| < 90^\circ$ else BACK, combined with left/right. This bearing reproduces the ReVSI ground-truth letter on all 290/290 relative-direction samples, so the residual error on this task is orchestration, not geometry.

Table A3 summarizes the full tool registry and which pipelines expose each tool.

Tool	Role (in/out)
<code>detect_object_3d</code>	name $\rightarrow$ camera-space OBB (predictor-agnostic backend)
<code>project_box_to_world</code>	name $\rightarrow$ world-space OBB (pose lift)
<code>calculate_object_distance</code>	two names $\rightarrow$ surface distance (m)
<code>calculate_object_size</code>	name $\rightarrow$ longest dim (cm)
<code>relative_direction</code>	three names + mode $\rightarrow$ 3-/4-way label
<code>detect_object</code>	name $\rightarrow$ 2D bbox (Grounding DINO)
<code>estimate_depth</code>	bbox $\rightarrow$ metric depth (Depth Pro)
<code>project_to_world</code>	bbox+depth $\rightarrow$ 3D world point
<code>calculate_distance</code>	two points $\rightarrow$ center distance (m)
<code>estimate_size_from_bbox</code>	bbox+depth $\rightarrow$ apparent size (cm)

Table A3. Tool registry. Top block: 3D OBB tools (used by the `bbox_3d.*` and `autonomous_3d` pipelines). Bottom block: the 2D + depth contrast tools (`gdino_depthpro.*`). Every tool takes object *names*; geometry is cached host-side.

Table A4 groups these tools into the four core pipelines (toolsets) referenced from Sec. 3, one per task, plus the 2D + depth baselines for absolute distance and object size.

D. Qualitative Agent Traces

Listing A4 shows a complete successful absolute-distance trace (tv $\leftrightarrow$ wall picture): the model follows the recipe exactly (detect, lift, detect, lift, measure), and the surface-distance tool returns 3.05 m against a 3.20 m ground truth (MRA 1.00), while the naive center-to-center distance would have been 3.34 m. Listing A5 shows the dominant relative-direction failure mode: the model calls *zero* tools, reasons directly from the frames, and guesses the wrong quadrant, which is the “discipline” gap analyzed in Sec. 4.

Listing A4. Success trace (`bbox_3d_abs_dist`, GT backend). CoT abbreviated; tool calls and returns verbatim. GT 3.2 m, prediction 3.05 m (MRA 1.00).

```
Q: Measuring from the closest point of each
   object, what is the direct distance between
   the tv and the wall picture?

[1] CoT: follow the sequence; start with the tv.
    -> detect_object_3d(object_name: "tv")
    <- {center_cam: [-1.17, 0.36, 1.74],
        extent: [0.04, 0.89, 0.51]}

[2] CoT: lift the tv box to world space.
    -> project_box_to_world(object_name: "tv")
    <- {world_center: [-2.96, 1.32, 0.78],
        extent: [0.04, 0.89, 0.51]}

[3] -> detect_object_3d(object_name: "wall picture")
    <- {center_cam: [0.87, -0.22, 1.43],
        extent: [0.77, 0.05, 0.80]}

[4] -> project_box_to_world(object_name:
    "wall picture")
    <- {world_center: [0.36, 0.94, 0.79],
        extent: [0.77, 0.05, 0.80]}

[5] CoT: both in world space; measure the distance.
    -> calculate_object_distance{
        object_a: "tv", object_b: "wall picture"
    }
    <- {distance_m: 3.05, center_distance_m: 3.34}

[final] Answer: 3.05
```

Listing A5. Failure trace (`bbox_3d_reldir`, GT backend): a zero-tool guess. The bearing tool would have returned LEFT (A); the model answered RIGHT (B) from the frames alone.

Task	Pipeline (Toolset)	Geometry Representation	Min. Steps	Median Steps
Abs. Distance	gdino.depthpro.abs_dist	2D bbox + Depth (center-to-center)	7	7
Abs. Distance	bbox_3d.abs_dist	3D OBB (surface-to-surface)	5	5
Object Size	gdino.depthpro.size	Apparent 2D projection	3	3
Object Size	bbox_3d.size	3D OBB extent (rigid invariant)	2	2
Rel. Distance	bbox_3d.reldist	3D OBB (nearest-instance surface)	14	16
Rel. Direction	bbox_3d.reldir	3D OBB (egocentric bearing angle)	7	8

Table A4. The four core toolsets, plus the 2D + depth baselines. Direct 3D box pipelines compute surface-to-surface relationships and rigid invariants. Median Steps (Sec. F; GT backend for the 3D pipelines, real detector for the 2D) equals Min. Steps on every numeric pipeline and exceeds it only on the two multiple-choice tasks, a benign overhead: a premature bearing call that errors and is retried (rel. direction), and redundant distance calls (rel. distance).

Q: If I am standing by the wall picture and facing the radiator, is the dresser to my left, right, or back?
Options: A. left B. right C. back (GT: A)

[final] CoT (no tool call): "wall picture visible above the bed headboard; radiator on the far wall; the dresser is off to the side ... it should be to my right."
Answer: B (WRONG -- 0 tools called)

E. Perception and Quantitative Visualizations

Perception. Figure A1 compares the camera-space 3D boxes produced by each `detect_object_3d` backend on the same object, making the perception bottleneck of Sec. 4 visible: the OVMono3D box is oversized and offset and the Cube R-CNN box drifts onto a neighboring object, while WildDet3D closely tracks the GT oracle. Figure A2 shows the intermediate 2D + depth signals of the `gdino.depthpro.abs_dist` pipeline.

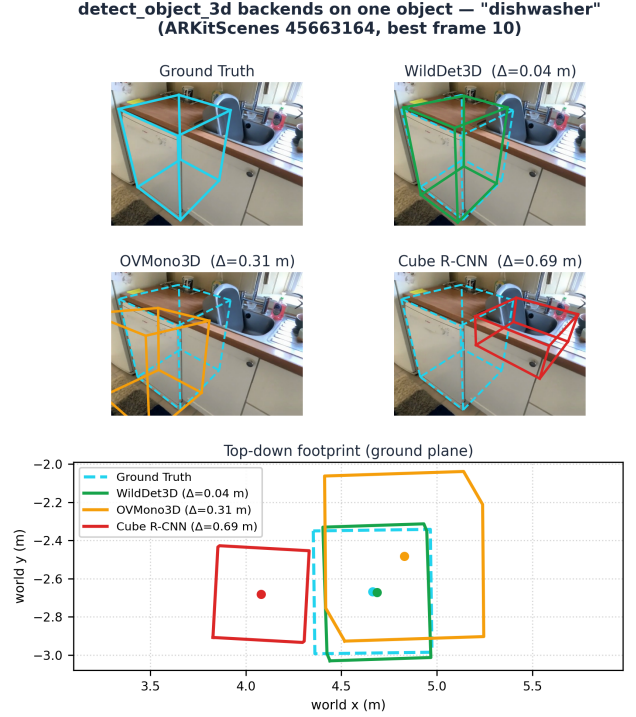


Figure A1. Detector backends behind `detect_object_3d` on one object (dishwasher, ARKitScenes 45663164, best frame 10). *Top four panels:* each backend's predicted 3D box (solid) with the ground-truth box (dashed cyan) reprojected onto the best frame; Δ is the world-space center error. *Bottom:* the same boxes as top-down ground-plane footprints. WildDet3D ($\Delta=0.04$ m) nearly coincides with GT, OVMono3D (0.31 m) is oversized and offset, and Cube R-CNN (0.69 m) drifts onto the sink. The tool-calling chain is identical across backends; only the box origin changes, so this gap is the perception bottleneck of Sec. 4.

2D + depth pipeline intermediates (gdino_depthpro)
ARKitScenes 41069042 — center-to-center 3.10 m vs. surface GT 2.4 m

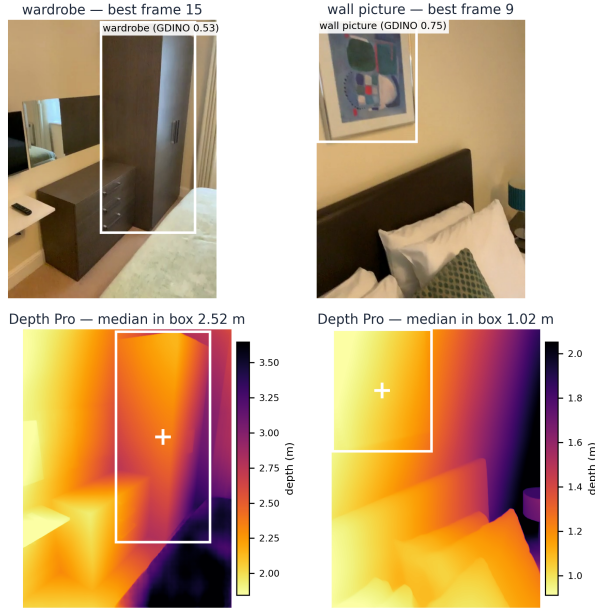


Figure A2. Intermediate signals of the 2D + depth pipeline (gdino_depthpro_abs_dist) on a wardrobe/wall-picture distance query (ARKitScenes 41069042). *Top*: each object’s best frame with its Grounding DINO 2D box. *Bottom*: the metric depth map Depth Pro predicts for that frame (bright = near, each map scaled to its own range), with the same box and the sampled center pixel (“+”); the pipeline keeps only the median depth inside the box (2.52 m for the wardrobe, 1.02 m for the wall picture). Back-projecting the two box centers yields a center-to-center distance (3.10 m) that overshoots the surface-to-surface ground truth (2.4 m), the systematic bias the OBB pipeline corrects. Depth Pro also reads the bedspread in the lower right as *far* (dark) although it is the nearest surface: monocular depth degrades on large texture-less regions, the weakness behind the error tail in Fig. A4.

Quantitative plots. Figure A3 and Fig. A4 give the same 401 absolute-distance questions in accuracy and in metric error; together they are the depth blow-up argument of Sec. 4. The 2D + depth baseline has a small median error but a heavy tail that inflates the MAE, whereas the OBB pipelines are stable. Figure A5 shows the per-sample MRA distributions; Fig. A6 shows why the 2D baseline is unreliable on object size.

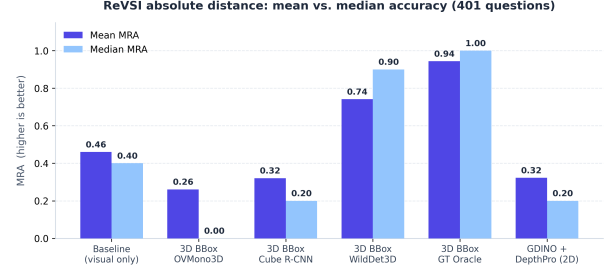


Figure A3. Mean vs. median MRA across every absolute-distance pipeline (401 questions); the gap between the bars measures tail sensitivity. Only WildDet3D beats the no-tool baseline: OVMono3D (0.26), Cube R-CNN (0.32) and the 2D+depth pipeline (0.32) all land below it (0.46), so a poor box is worse than no box. OVMono3D’s median is 0.00, i.e. more than half its answers are off by over 50%. WildDet3D (mean 0.74, median 0.90) approaches the GT oracle (0.94/1.00).

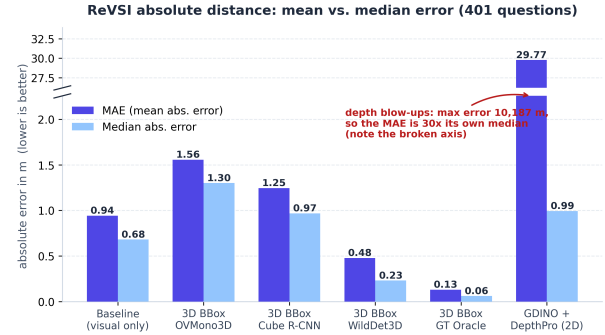


Figure A4. The same pipelines in metric *error*, which corroborates the MRA ranking. The weak 3D backends are worse than the no-tool baseline (MAE 1.56 m for OVMono3D and 1.25 m for Cube R-CNN, against 0.94 m), while WildDet3D (0.48 m) and the GT oracle (0.13 m) sit far below it. Every OBB pipeline stays bounded (max error ≤ 6.2 m), so their means are trustworthy. The 2D+depth pipeline is the exception: monocular depth blows up on boundary boxes to a worst error of 10 187 m, leaving an MAE (29.8 m) thirty times its own median (0.99 m).

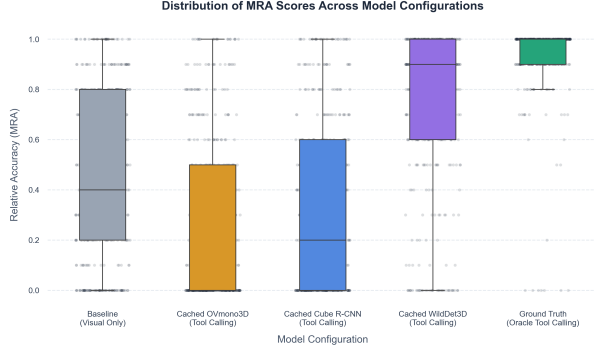


Figure A5. Per-sample MRA distributions on absolute distance (401 questions) for the no-tool visual baseline and each `detect_object_3d` backend. The GT oracle concentrates at 1.0 (median 1.00) and WildDet3D tracks it (median 0.90), whereas OVMono3D and Cube R-CNN pile up at zero: their medians (0.00 and 0.20) sit *below* the no-tool baseline’s (0.40), so over half of OVMono3D’s answers are more than 50% off. Tool augmentation only pays off once the detector is strong enough.



Figure A6. Object size from a 2D box (`gdino.depthpro.size`) on six ReVSI objects: the Grounding DINO box (red) and mask (cyan) with the longest image-plane axis (yellow). Panel titles give GT and both back-projected estimates in cm, mask-based and box-based (`bbox`, the reported variant). Back-projection sees only the image-plane dimensions, so it is accurate when the longest axis lies in that plane (kitchen sink 104 → 103, couch 165 → 168) and fails in two ways: the box covers only part of the object (floor lamp 195 → 80: only the shade is grounded), or it lands on the wrong surface (dresser 90 → 118). This variance, not a constant bias, caps the 2D size pipeline at 0.34 MRA.

F. Tool-Call Step Analysis

Figure A7 compares the *minimal correct chain* against the tool calls the orchestrator actually made (GT backend, so the counts isolate orchestration from perception). The minimal chain is structural, $2n_{\text{obj}} + n_{\text{meas}}$ for the OBB pipelines (one `detect` and one `project` per object, plus the measurements), dropping the `project` step for object size: 2 calls for size, 5 for absolute distance, 7 for relative direction, and 14 for relative distance (reference + 4 candidates). Three patterns stand out.

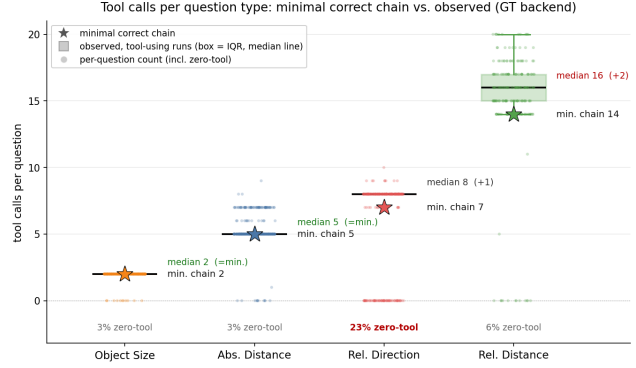


Figure A7. Minimal correct chain vs. observed tool-call count per question type (GT backend). Stars mark the minimal chain ($2n_{\text{obj}} + n_{\text{meas}}$, structural per question); boxes summarize the tool-*using* runs (median line, IQR), and jittered points show every run including the zero-tool band (annotated %). Numeric tasks run the minimal chain exactly; relative direction runs +1 (a premature `relative_direction` call that errors and is retried) and relative distance +2 (redundant distance calls). The 23% zero-tool rate on relative direction is the orchestration *discipline* gap, separate from this overhead.

Numeric tasks run the minimal chain exactly. Object size sits at 2 (412/423 samples) and absolute distance at a median of 5. This near-zero orchestration overhead is what lets GT-backend accuracy sit within 3 points of the tool ceiling (Sec. 4).

The multi-object MC tasks carry a small, systematic overhead. Relative direction runs +1: 198 of the 223 engaged runs call `relative_direction` *twice*, once prematurely (before the boxes are lifted, which returns the “project first” error) and again after projecting. Relative distance runs +2, from redundant `calculate_object_distance` calls beyond the 4 it needs. Both are benign, since the answer is still correct, but sequencing is not tight on the longer chains.

The discipline failure is a separate zero-tool spike. Every task carries a band of runs that call *no* tool and answer from the frames: 2.6–5.6% on the numeric and relative-distance tasks, but 23% (67/290) on relative direction. Because that unaided guess is no better than chance (25.9% visual vs. 29.3%, Sec. 4), it is this spike rather than the sequencing overhead that caps relative-direction accuracy at 80% against a 100% geometric ceiling.

G. Relative-Direction Subtype Breakdown

Table A5 breaks relative-direction accuracy down by subtype. The GT oracle is stable across subtypes (73.6–84.1%); WildDet3D trails it by at most 8 pt everywhere except the “backward hard” cases (84.1 → 68.1%), the doubly boundary-sensitive ones: they are 4-way quadrant questions, so center noise can flip the target across the 90° front/back split, and they use the `away` facing mode, which negates `f`, so noise on the *viewer* center swings the bearing too (Sec. C). Even

so, the overall oracle gap is only 6.6 pt against a 25-pt gap on relative distance. The bearing reads object *centers* alone and never their extents, which makes it far less detector-sensitive: swapping GT boxes for WildDet3D costs the tool-only ceiling 11 pt (100.0 $\rightarrow$ 89.0%), where the same swap costs relative distance 25.6 pt (94.9 $\rightarrow$ 69.3%). The larger share of the remaining error on this task is therefore tool-use discipline rather than perception.

Subtype	N	GT Oracle	WildDet3D
Backward Easy (3-way)	53	79.2%	75.5%
Backward Hard (4-way)	69	84.1%	68.1%
Forward Easy (3-way)	96	82.3%	74.0%
Forward Hard (4-way)	72	73.6%	76.4%
<i>Overall</i>	290	80.0%	73.4%

Table A5. Relative-direction accuracy by subtype (“easy” = 3-way left/right/back; “hard” = 4-way quadrant; “backward” = the away facing mode). The GT oracle averages 6.1 tool calls per question over all runs, or 8.0 (median 8) over the runs that call any tool at all; the gap to the 100% geometric ceiling is the 23% zero-tool guessing rate.

H. Autonomous Probe: Tool Feedback and Prompting

The no-recipe probe of Sec. 4 strips the per-task recipe and leaves only the generic tool description of Listing A3: the model must route and order the tools itself. This section reports two runs, both on the GT backend. The first is a staged ablation on 80 questions (absolute distance and object size only) that varies *what the environment tells the model* and produced the tool-error strings we now ship (Tab. A6). The second is the four-task probe on 160 questions, which supplies the numbers reported in Tab. 4. They are separate runs over different task mixes and question samples, so their per-task numbers are not directly comparable; the four-task probe is the one we report.

Staged ablation (80 questions). *Stage 1 (terse errors)* exposes the tool errors of the original library. Calling `calculate_object_distance` before detecting and lifting the boxes returns only “No 3D box found for [tv, sink]”, which does not say what to do next, so the model retries the same call: 138 error results over 80 questions, and a mean of 5.0 calls where 3.5 suffice. This is the looping the main text reports, and it costs the most on distance (0.58 MRA), the task with the longest chain.

Stage 2 (actionable tool errors) rewrites every tool error into an actionable instruction naming the missing step, e.g. “No world-space box for [tv, radiator]. For each of these objects call `project_box_to_world` first.” Errors collapse to 36 and distance jumps to 0.88, but the prompt still never shows the model *when* a tool is needed: zero-tool guessing rises to 14/80, dragging size down to 0.74.

Stage 3 (few-shot order examples) appends Listing A6: one tool-order example per task, showing which tools to chain and in what order, never an answer. Both failure modes disappear at once (errors 13, zero-tool 0/80) and MRA reaches 0.93. Within this ablation the examples look decisive; the four-task probe below shows they are not.

Condition	MRA	Dist.	Size	Errors	Zero-tool
1. Terse tool errors	0.729	0.575	0.883	138	5/80
2. + actionable errors	0.800	0.878	0.723	36	14/80
3. + few-shot examples	0.933	0.938	0.928	13	0/80

Table A6. The staged ablation on tool feedback (80 questions: 40 absolute distance, 40 object size; GT backend). *Errors* counts tool results returned as errors across all 80 traces; *Zero-tool* counts runs that answered without calling a single tool. Better feedback removes the looping, and here the few-shot examples remove the remaining guessing; on the four-task probe (Tab. 4) they no longer help.

Listing A6. Few-shot block appended to the `autonomous_3d` hint (Listing A3) in stage 3. One tool-order example per task; no ground-truth answers and no per-question recipe.

```
Examples of tool choice+order (these show which
tools, NOT the answers):
- size: 'longest dimension of the chair?'
  -> detect_object_3d(chair);
      calculate_object_size(chair).
- distance: 'distance between tv and sofa?'
  -> detect_object_3d(tv);
      project_box_to_world(tv);
      detect_object_3d(sofa);
      project_box_to_world(sofa);
      calculate_object_distance(tv, sofa).
- rel-distance: 'which of X/Y/Z is closest to the
lamp?' -> lift the lamp and each candidate
      (detect_object_3d + project_box_to_world),
      calculate_object_distance(lamp, each),
      pick the smallest.
- rel-direction: 'standing by A facing B, where
is C?' -> lift A, B, C (detect_object_3d +
      project_box_to_world each),
      relative_direction(A, B, C).
```

Four-task probe (160 questions). Running all four tasks at once (40 questions each), with the rewritten tool errors in place, gives the results of Tab. 4 and confirms that routing itself is not the bottleneck. With the *generic* prompt alone (no recipe, no examples), the model calls the correct final tool on 98% (size), 100% (distance) and 90% (relative distance) of questions and scores 0.97 MRA, 0.94 MRA and 92.5% accuracy, matching its per-task numbers under the hardcoded recipe. Relative direction is the single exception (70% routed, 70.0% accuracy): on 30% of those questions it calls no tool at all, the same discipline failure that caps the recipe-driven runs (Sec. F).

The few-shot examples that were decisive in the staged ablation do *not* carry over. Repeating the four-task probe with the stage-3 few-shot block leaves size (1.00) and relative direction (75.0%) slightly better but drags absolute distance

to 0.89 and relative distance to 80.0%, below the generic prompt on both. Once the tools report their errors clearly, the in-context examples add nothing to routing and can bias the model toward the example’s chain on questions that need a different one. We therefore report the generic prompt as the headline autonomous condition, not the few-shot one.